

MLCVZoo: A Meta-Framework for Machine Learning-based Computer Vision algorithms

REDACTED FOR REVIEW

Abstract—We present the Machine Learning Computer Vision Zoo: MLCVZoo, a novel open source meta-framework that integrates different libraries and algorithm implementations from the field of Machine Learning based Computer Vision by providing a generic data and model Application Programming Interface (API). The API design focuses on generalizing functionalities that are shared across different Computer Vision domains, as well as domain specific fits between data structures and algorithm behavior. Users then only need to familiarize themselves with and develop against the MLCVZoo API once. By doing so, they are able to directly use and compare different algorithms from different libraries and implementations. This also eliminates the problem of a vendor lock-in, as users can change between algorithm libraries and implementations later in the development cycle without having to rewrite their software. All MLCVZoo modules are released at REDACTED_FOR_REVIEW.

I. MOTIVATION

Deep learning has been driving Computer Vision for about a decade and libraries have been created, discarded, and enhanced ever since [12]. They evolve around a wide range of research and application areas. Libraries like Tensorflow [4], PyTorch [20] and ONNX [2] are well known and often serve as the backbone of many algorithm implementations. Furthermore, there are several libraries aiming at providing building blocks for simplifying the development of new algorithms, as will be discussed in chapter III. During the selection process of a concrete algorithm for an application, users are almost always facing a speed-accuracy trade-off [11]. In addition, the field of Deep Learning-based Computer Vision algorithms is volatile, as can be seen when following the lists of state-of-the-art algorithms at [1]. Oftentimes, new algorithms are developed in different libraries or even standalone implementations, forcing users to adapt their application each time they find a better solution that optimizes their speed-accuracy trade-off. Overall, there is lack of having a generic API that generalizes the interfaces of well established Computer Vision domains, such as Image Classification, Object Detection or Instance Segmentation, independent of the library in which the implementations are developed in. For example, it is a well known fact that every Object Detection algorithm will produce Bounding Boxes in some data format.

This is where the MLCVZoo (Machine Learning Computer Vision Zoo) comes into play. It supports the development of Computer Vision applications by wrapping various libraries and algorithms via a unifying API. In chapter IV the design and key benefits of this API are discussed. The focus lies on defining generic features that are shared across all major Machine Learning driven Computer Vision domains. By doing so, it eliminates the effort of integrating frameworks individually

with every new library, and allows to build Computer Vision applications against a single API.

II. USE CASES

Logistics processes are continuously improved and adapted to new requirements. Decisions on how and where to optimize are based on the analysis of existing production processes. Therefore, there is a continuous demand of identifying new use cases where Computer Vision can help to provide data that maximizes the benefit of such analysis and in turn optimize relevant business Key Performance Indicators (KPIs). In following, several use cases are discussed that we developed together with our research and industry partners, where Computer Vision was utilized to achieve the aforementioned benefits.

Transport Management: The objective is to automate the gathering of necessary information for optimizing land transport operations. As an initial application, the system scans all trucks entering and exiting a pilot branch, extracting their license plate and potential Intermodal Loading Unit (ILU) code from video data. This information facilitates various process efficiencies, including automated yard access control, real-time allocation of loading and unloading docks, and touchless inventory management of equipment on the premises.

Tracking of logistic entities in a warehouse: Business managers in the field of intralogistics are striving for analytical methods on the basis of which an increase in the transparency of warehouse processes can be achieved. Necessary for the transparency of warehouse processes is the analysis of the flow of goods and the movement profiles of the objects in the warehouse. This makes it possible to compare the processes planned in the warehouse management system with the real world processes. In this context, the application of Object Tracking algorithms from the field of Computer Vision is very promising. One major input for Object Tracking are Bounding Boxes derived from Object Detection algorithms [22].

Automatization of a portal crane - Automodal: The project was concerned with the development of a prototype of an automated portal crane that is needed as a step for a continuous automatization of trimodal terminals [REDACTED FOR REVIEW]. One of the key factors was the reliable detection of humans for ensuring personnel safety, since the use case was focusing on non fenced off areas. To achieve this, a software system was developed that bundles all necessary sensor readings, as well as implementing the interfaces to the systems that are controlling the crane itself [3]. For automated detection of humans using color cameras, Object Detection algorithms have been applied. Besides having a reliable detection, the inference time was crucial to allow a detection in

real time. As a solution, a dedicated MLCVZoo component was integrated to build the overall software solution against a generic API, while allowing to use algorithms which maximize the speed-accuracy trade-off.

As shown above, there are several promising and established use cases for Machine Learning based Computer Vision algorithms in the field of logistics. It can be observed, that all use cases have to solve the problem of optimizing the speed-accuracy trade-off for the concrete application. Furthermore, it appears that there are analogies between different libraries and algorithms, and the lack of a generic API that generalizes the basic functionalities that are shared across all Machine Learning based Computer Vision methods like Image Classification, Object Detection, (Instance) Segmentation and Optical Character Recognition (OCR). These observations were the initial motivation for implementing such an API in the form of the MLCVZoo. By having such an API at hand, applications can be built focusing on the use case itself, leaving the implementation details for attaching and using Machine Learning algorithms to specific modules of the MLCVZoo.

III. RELATED WORK

The core idea of a unified API for different Deep Learning algorithms is not new. Popular projects related to the MLCVZoo such as MMDetection [6], Detectron2 [25] and the TensorFlow Object Detection API [4], aim to implement popular neural network architectures in a unified way with the use of Machine Learning specific libraries such as PyTorch (MMDetection and Detectron2) and TensorFlow (TensorFlow Object Detection API).

The MMDetection library is part of the OpenMMLab ecosystem and mainly dedicated to Object Detection and Instance Segmentation. It was designed to provide a high quality code base and a unified benchmark to easily compare different algorithms and features a modular design which allows users to combine different backbones and detection-heads. [6]

In addition to MMDetection, there are other libraries in the OpenMMLab ecosystem that have been developed for specific tasks within the field of Computer Vision. Examples are MMOCR for OCR and MMPose for Pose Recognition. [19]

The TensorFlow Object Detection API was originally implemented in 2015 to fairly compare SSD, R-CNN, and R-FCN in terms of runtime efficiency. It provides a unified training pipeline, which means that all implemented models can be trained in the same way. The basis for a training is a single configuration file, which additionally offers the user the possibility to configure the training according to his needs. Among other things, the architecture of the model, the learning rate and various training parameters can be configured in a unified manner. [10]

Detectron2 supports multiple use cases, namely: Object Detection, Instance Segmentation, Human Pose Prediction as well as Panoptic and Semantic Segmentation. Just like the previous libraries, it features a modular and extensible design. It is designed to give users the ability to test and implement new neural network architectures faster and provides an additional

software layer called Detectron2go which implements different features to bring trained models easier to production in the cloud or on mobile devices. [24], [25]

In addition to these libraries, there are a number of others that take similar approaches, such as SuperGradients [5] and HuggingFace [23].

Although the libraries mentioned above have minor differences, some important commonalities can be identified. These are of particular importance for the design of the MLCVZoo API and are highlighted below:

Unified API: The core idea of the presented libraries is to make a variety of algorithms available via a unified API. This includes data pre- and post-processing, the actual usage of the algorithms as well as the evaluation of different algorithms within one library. [6], [10], [24]

Modular and extensible design: The algorithms are implemented in a way, which provides users a high flexibility in combining different backbones and detection-heads [6], [10]. Additionally, the structure of the library facilitates implementing new algorithms and thus to extend the library [24].

Individual configurability: Each library enables users to individually configure the implemented algorithms to their needs. The central point for this is a configuration file with which users can customize the respective algorithm. [10], [17], [25]

In summary, there are already some libraries that bundle different Computer Vision algorithms into a unified API. The MLCVZoo however intends to provide a unified API not only across different Computer Vision algorithms within a single library but across different Computer Vision libraries. In essence, the MLCVZoo is a *meta-framework*, intended to be a library over Computer Vision libraries making different Computer Vision algorithms, as well as different implementations of those algorithms transparently interchangeable. Its main goal is to provide the aforementioned properties across already existing Computer Vision libraries.

IV. ARCHITECTURE

As mentioned, the benefit of having a unified API for Computer Vision algorithms, is the elimination of effort of integrating libraries or single algorithm implementations individually with every new Computer Vision project. Therefore, the MLCVZoo API consists of a modular set of components, conceptualised around wrapping different libraries and algorithm implementations. Emphasis is put on extensibility to reduce effort necessary for adding additional Computer Vision libraries to the MLCVZoo, as well as allowing users to pick and use only necessary parts of the MLCVZoo to keep dependencies and constraints for software projects low. In the following, the key concepts around the construction of the MLCVZoo API, as well as the different modules that build up the MLCVZoo are discussed.

A. Key Concepts

The MLCVZoo picks up on the advantages described in chapter III by taking the idea of a unified API to a higher level.

Next, the key concepts behind the design of the MLCVZoo that facilitate this will be examined.

Common abstraction for heterogeneous library APIs:

The MLCVZoo acts as a meta library and therefore abstracts concepts to be universally applicable across Computer Vision algorithm implementations. For now, it is focused on Image Classification, Object Detection, Instance Segmentation and OCR. Instead of providing a library of module blocks (e.g. Bounding-Box Head, Classification Backbone) with which different algorithm architectures can be implemented, the MLCVZoo abstracts at a higher hierarchy level by wrapping whole libraries and implementations of single algorithms. This results in a benefit for the user by still having a uniform API at their disposal - in this case, however, across library boundaries. If an application needs to apply algorithms of different libraries (e.g. MMDetection and Detectron2) to implement its Computer Vision use case, it can now be developed against one single API. Developing an application against a single, implementation agnostic API has the benefit of not having to commit to a single Computer Vision library from the beginning and therefore avoiding a vendor lock-in. As mentioned before, this is relevant as the field of Machine Learning driven Computer Vision algorithms and libraries is volatile.

Figure 1 shows the comparison between an integration of multiple different libraries into an application versus integrating the corresponding components of the MLCVZoo. The blocks marked with an asterisk are not yet integrated into the MLCVZoo and subject to ongoing development, but illustrate the vision of having a joint API.

Modular and extensible design: Overall, the MLCVZoo has a modular and extensible design to serve different needs of users as well as being flexible to integrate new features of libraries or writing modules for wrapping completely new libraries. The different modules are designed to be independent so that new libraries can be integrated regardless of the existing modules. Since different implementations can have conflicting dependencies, the choice of independent modules is important for a stable software setup. However, the MLCVZoo tries to impose as few restrictions as possible to allow combining its different modules as much as possible. Besides having a modular setup, the design of the API allows that it can be extended with new features easily if desired. For the time being, it focuses on the basic functionalities that are shared across the already integrated libraries and algorithms and the corresponding Computer Vision domains.

Configuration Centered Design: As mentioned in the previous sections, the MLCVZoo targets Computer Vision researchers and practitioners which apply and select ML models to their needs. Since the hyper-parameter optimization of algorithms is a crucial task, the design of the MLCVZoo API is conceptualized around configuration components. We use *YAML* files for configuration to separate parameters from code and allow a documented comparison on algorithms based on different parameterizations. For every library integration there are generic configuration aspects that are handled via the MLCVZoo, as well as library specific sections. These library specific sections are designed to be close to the original

implementation, because the abstraction of the MLCVZoo in regard to the original implementation should be minimized in order to maximize the usability. This allows the user to be guided by the original documentation as well as potential bug reports of the implemented library, which increases usability.

B. MLCVZoo Modules

The MLCVZoo consists of several modules that serve different purposes. However, they can be grouped into two categories: Modules that serve as wrapper implementations for distinct Computer Vision libraries and algorithms, and modules that form the basis of its API as well as offering additional functionalities. In the following, we discuss the architecture of these modules and their intended purposes. A detailed arc42 documentation about all the described components can be found in the repository.

MLCVZoo API and basic functionality: The *mlcvzoo-base* module defines the API of the MLCVZoo, as well as components for handling generic data preparation and evaluation tasks. The API includes the definition of base classes and interfaces for handling different kinds of Computer Vision algorithm domains, as well as the definition of data structures that are used across the MLCVZoo modules. From an algorithm implementation perspective, the *Model* class builds the generic base for every Computer Vision algorithm that should be implemented in the MLCVZoo. It provides the core features every model implementation should provide. Extending this class, the *mlcvzoo-base* API defines a specific, abstract model class for every major Computer Vision algorithm domain. For now this includes: Image Classification, Object Detection, Instance Segmentation and OCR. This is motivated by the fact that every major algorithm domain has a dedicated task, e.g. for Object Detection this is the detection of Bounding Boxes in a given image. Therefore, there are dedicated data structures that are produced and consumed by the corresponding models. Overall, the generic model formulation together with the defined data structures comprise the core of the MLCVZoo API. Wrapper implementations will choose one of these domain specific model classes as their base class to define their specific prediction behavior.

Apart from its prediction behavior, one key feature of any MLCVZoo model is to define a configuration class. This is intended to contain every relevant parameter of the model that should be configurable from the outside of the code. All configuration is bound by and inherits from the *ModelConfiguration* class of the generic *Model* class. To also provide access to additional configuration parameters specific to certain algorithms or Computer Vision libraries wrapper implementations define specific configuration subclasses that extend the MLCVZoo configuration. This results in a single, common configuration controlling not only the behaviour of the MLCVZoo and common functionality, but also functionality specific to any single algorithm or Computer Vision library. Leveraging the central configuration component together with the API definitions, the *mlcvzoo-base* module provides the MLCVZoo Model Registry which can instantiate and run any algorithm from any wrapper implementation the user has

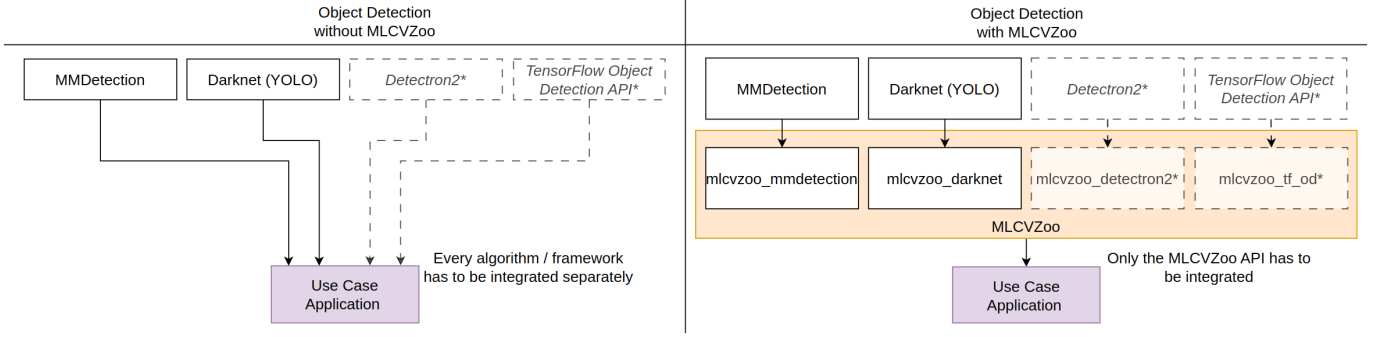


Fig. 1. MLCVZoo key benefit for a use case that is using Object Detection algorithms

installed, selected and parameterized purely by configuration, and with minimal effort.

Apart from the core features, a wrapper implementation can extend its behavior by choosing to implement one or more of the defined API Interface classes. The trainable interface is associated with any model for which the term “training” can be applied and implemented in the MLCVZoo. This usually means the training of a neural network. Algorithms that rely on a neural network can implement the NetBased interface. Typically, any neural network can store and restore its weights. A model that inherits from the Classifiable interface states that its prediction output is representing some kind of class instance. A class in this context means a real world entity such as “person”, “car”, “truck”, “cat”, etc.

As mentioned, the *mlcvzoo-base* module provides some data preparation components, in addition to the API. One key component is the AnnotationClassMapper that can be utilized for translating class names of annotation files to model IDs and vice versa. This is needed when parsing annotation data for a model training or when the output of a neural network should be decoded. It also provides functionality to reduce a given mapping of annotations to model IDs by mapping several annotations to a single model ID or vice versa, increasing reusability of annotation files for different models. As an example, annotations defining “car” as well as “truck” can be mapped to “vehicle” for models not differentiating between those two classes. This allows to adapt or modify class mappings and annotation sets by configuration without having to use annotations that match models exactly. Furthermore there are annotation parsers and writers for the most common Computer Vision dataset formats, namely: COCO [15], CVAT, LabelStudio (for Object Detection), PascalVOC [9], MOTChallenge 2015, 2016, 2020 [14], [16], [8].

Besides its data-preparation components, the *mlcvzoo-base* module provides algorithm specific evaluations. Currently however, this is only implemented for Object Detection algorithms.

Utilities & Services: The *mlcvzoo-util* module includes everything that is not intended to be extended by other MLCVZoo modules. It consolidates utility tools, to ease model training, evaluation or timing and other functionalities primarily delivered in the form of CLI tools. The *mlcvzoo-service* module is a REST service to train arbitrary MLCVZoo

models. It leverages the aforementioned MLCVZoo Model Registry to provide a quick and easy way to train models via the MLCVZoo without having to care about implementation details.

Integrated ML Libraries & Algorithms: Integrating different Computer Vision libraries into the MLCVZoo is done by implementing dedicated MLCVZoo wrapper modules. At the time of writing, there are a couple of libraries already integrated, which are shown in table I. A wrapper module essentially consists of a model wrapper class that inherits from a base model class and the interfaces that apply to this wrapper, as well as a configuration class to load information from a configuration YAML file. After implementing the inherited methods the new module can be used as a generic replacement for any use case that fits to its base class.

Algorithm Type	Wrapped Library or Model	MLCVZoo module
Classification	Tensorflow XceptionNet [21]	mlcvzoo-tf-classification
	Tensorflow Custom Block Net	
Object Detection	Darknet [7]	mlcvzoo-darknet
	MMDetection [6], [17]	mlcvzoo-mmdetection
Segmentation	MMDetection	mlcvzoo-mmmmdetection
OCR	MMOCR [13], [18]	mlcvzoo-mmocr

TABLE I
INTEGRATED LIBRARIES & ALGORITHMS

The *mlcvzoo-tracker* module is a component original to the MLCVZoo that implements the main parts of the Object Tracker published in [22]. It is a generic, online, Single-Image Multi-Object Tracking component, which takes Bounding Boxes and images as input to perform the tracking operations.

V. EVALUATION

In this chapter, the developed MLCVZoo API is qualitatively evaluated from different points of view that are of interest for practical use.

A. Evaluation scale

The MLCVZoo is designed to reduce effort required to use different Computer Vision libraries, under the premise, that different Computer Vision libraries are to be used or evaluated either in a single or across multiple projects. The evaluation of the MLCVZoo is therefore made in comparison

with the direct use, either simultaneously or consecutively, of several individual Computer Vision libraries. Using the MLCVZoo can therefore be better, equal, or worse than using the individual Computer Vision libraries directly. The following aspects are crucial for the evaluation:

B. Uniform API

The key to reducing effort is a uniform API, so that it is not necessary to learn a new API for every library. Two aspects are essential here: First, integrating different Computer Vision libraries via the MLCVZoo should not differ greatly between different libraries, and second, the specific functionality of the individual libraries should still be usable.

The MLCVZoo achieves this by, on the one hand, enabling a uniform type of configuration by means of YAML files that are always structured in the same way and, on the other hand, providing an identical API (e.g. `train()`, `predict()`, `save()`, `restore()` methods) for all Computer Vision libraries in the same domain. However, the configuration of the models of the various Computer Vision libraries, although always structured in the same way, can still make specific functionality (e.g. certain floating point precision) of certain Computer Vision libraries usable.

The MLCVZoo reduces the integration effort of a Computer Vision library, since the identical API can be used for all domain-specific libraries. At the same time, the user does not have to forgo any specific functions of the individual Computer Vision libraries, since these are taken into account by the model-specific configuration of the MLCVZoo. All in all, the MLCVZoo therefore offers advantages over the direct use of the Computer Vision libraries.

C. No significant additional runtime overhead

In principle, every reduction in developer's effort through abstraction is offset by an increase in effort through additional computing time, due to more code being executed. In the case of MLCVZoo, however, in most use cases no significant portion of the computation time is spent in MLCVZoo itself, but in the integrated Computer Vision libraries, since MLCVZoo, simplified, merely calls the functions of the underlying library in the correct manner and order. Training and inference are considered below as obvious use cases.

For inference, a distinction must be made between two cases. For particularly time-critical use cases, which are at the same time characterized by low available computing capacities (e.g. real-time inference on embedded boards), the MLCVZoo is only suitable to a limited extent due to its additional abstraction. However, these special cases often require a tighter integration of hardware, software and Machine Learning components anyway (e.g. with more efficient C++ code without any abstraction overhead). The majority of use cases, however, are characterized by less time-critical requirements and/or higher computing capacities, so that the already small additional effort due to abstraction of the MLCVZoo is not significant here. During training, on the other hand, the volume of data to be processed is so large that the additional operations performed by the MLCVZoo are negligible. For training, the

additional effort is therefore also not a disadvantage compared to the direct use of the Computer Vision libraries.

In terms of the evaluation scale, the MLCVZoo is therefore equally well suited as the direct use of the Computer Vision libraries.

D. Additional tools

For training of Machine Learning based Computer Vision models, there are typically recurring activities that need to be performed for each training. These include activities from the areas of data pre-processing, evaluation, logging, etc. The MLCVZoo provides tools to help automate these activities and thereby reduce manual effort on the part of the user. The tools are designed to be used for all integrated Computer Vision libraries.

In relation to these activities, the following disadvantages can occur when using the libraries directly:

- The libraries to be used each already offer their own tools with an adequate range of functions. In this case, however, a learning curve is required for each library. This increases with the number of libraries used.
- The libraries to be used each already offer their own tools, but with insufficient functionality. In this case, an existing tool must be adapted. The adaptation of a tool must be carried out separately for each library. A new development may be required for missing tools.
- Custom tools already exist on the user side, but they are incompatible with one or more libraries. The adaptation of a tool must then be done for each of the libraries to be used.

The uniformly provided tools of the MLCVZoo offer the following advantages over this:

- The provided tools can be used for any integrated library and are used in the same way.
- A tool requires a one-time familiarization effort. Afterwards, it can be used as often as desired and in the same way for each library.
- If a tool for an activity is missing, it has to be developed only once. It can then be used for all libraries. Similarly, pre-existing tools have to be modified only once to work with all libraries integrated by the MLCVZoo.

Related to the evaluation scale and based on the advantages and disadvantages presented, the use of the MLCVZoo is therefore better than the individual use of the Computer Vision libraries.

E. Summary

In summary, the following evaluation results can be noted for the MLCVZoo: The MLCVZoo requires a one-time training effort to learn the API and the tools provided. Subsequently, all integrated libraries can be used in the same way. Apart from the special cases presented, the MLCVZoo does not lead to a significant additional computational effort which would impair the efficiency of the underlying libraries. From the user's point of view, the MLCVZoo is particularly suitable if one wishes to use several Computer Vision libraries.

With an increasing number of libraries used, the benefit of the MLCVZoo increases. This benefit is further increased if the user wishes to make use of tools that relieve him of repetitive activities in the context of model training. The limiting factor of the MLCVZoo is the abstraction overhead. Even if this can be neglected for a large part of the use cases, the abstraction overhead can be a disadvantage in the special cases presented. For these special cases, users should consider using the Computer Vision libraries directly.

In conclusion, the main advantage over the use of individual libraries lies in the uniform API and the additional tools, which reduce both the familiarization effort and the manual effort of the user. This reduction in effort is accompanied by additional runtime overhead, which may be a disadvantage for time-critical inference. Apart from this special case, the MLCVZoo appears to its authors to be at an advantage to the individual use of different Computer Vision libraries.

VI. SUMMARY

The MLCVZoo is a user centered meta-framework that simplifies usage of Machine Learning driven Computer Vision algorithms. Motivated by different use cases in the domain of logistics, the framework is designed to provide a unified API that allows a robust integration of these algorithms into user applications. Users are able to work with different Computer Vision libraries, while they only have to familiarize themselves with a single API once. We reflected the usage of the MLCV-Zoo compared to using several libraries or single algorithm implementations, which resulted in an advantage of using the MLCVZoo. There is generally no significant computational overhead in runtime, but a clear benefit from the software architecture and usability perspective.

ACKNOWLEDGMENT

REDACTED FOR REVIEW

REFERENCES

- [1] “Browse state-of-the-art,” <https://paperswithcode.com/sota>, accessed: 2023-07-04.
- [2] “Open neural network exchange (onnx),” <https://github.com/onnx/onnx>, accessed: 2023-07-04.
- [3] “Sensor and neural data platform (sand),” <https://gitlab.com/sand7/sand>, accessed: 2023-06-30.
- [4] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, “Tensorflow: A system for large-scale machine learning,” in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI’16. USA: USENIX Association, 2016, p. 265–283.
- [5] S. Aharon, Louis-Dupont, Ofri Masad, K. Yurkova, Lotem Fridman, Lkdci, E. Khvedchenya, R. Rubin, N. Bagrov, B. Tymchenko, T. Keren, A. Zhilko, and Eran-Deci, “Super-gradients,” 2021. [Online]. Available: <https://zenodo.org/record/7789328>
- [6] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Xu, Z. Zhang, D. Cheng, C. Zhu, T. Cheng, Q. Zhao, B. Li, X. Lu, R. Zhu, Y. Wu, J. Dai, J. Wang, J. Shi, W. Ouyang, C. C. Loy, and D. Lin, “Mmdetection: Open mmlab detection toolbox and benchmark,” *CoRR*, vol. abs/1906.07155, 2019. [Online]. Available: <http://arxiv.org/abs/1906.07155>
- [7] Darknet Contributors, “YOLOv4, Scaled-YOLOv4, YOLO - Neural Networks for Object Detection (Windows and Linux version of Darknet),” <https://github.com/open-mmlab/mmdcr>, 2021, accessed: 2022-12-07.
- [8] P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler, and L. Leal-Taixé, “Mot20: A benchmark for multi object tracking in crowded scenes,” *arXiv:2003.09003[cs]*, Mar. 2020, arXiv: 2003.09003. [Online]. Available: <http://arxiv.org/abs/1906.04567>
- [9] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL Visual Object Classes Challenge 2012 (VOC2012) Results,” <http://www.pascal-network.org/challenges/VOC/voc2012/workshop/index.html>.
- [10] J. Huang, V. Rathod, C. Sun, M. Zhu, A. Korattikara, A. Fathi, I. Fischer, Z. Wojna, Y. Song, S. Guadarrama, and K. Murphy, “Speed/accuracy trade-offs for modern convolutional object detectors,” *CoRR*, vol. abs/1611.10012, 2016. [Online]. Available: <http://arxiv.org/abs/1611.10012>
- [11] —, “Speed/accuracy trade-offs for modern convolutional object detectors,” Conference: 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 07 2017, pp. 3296–3297.
- [12] L. Jiao, F. Zhang, F. Liu, S. Yang, L. Li, Z. Feng, and R. Qu, “A survey of deep learning-based object detection,” *CoRR*, vol. abs/1907.09408, 2019. [Online]. Available: <http://arxiv.org/abs/1907.09408>
- [13] Z. Kuang, H. Sun, Z. Li, X. Yue, T. H. Lin, J. Chen, H. Wei, Y. Zhu, T. Gao, W. Zhang, K. Chen, W. Zhang, and C. L. Zitnick, “Mmcocr: A comprehensive toolbox for text detection, recognition and understanding,” *arXiv preprint arXiv:2108.06543*, 2021.
- [14] L. Leal-Taixé, A. Milan, I. Reid, S. Roth, and K. Schindler, “MOTChallenge 2015: Towards a benchmark for multi-target tracking,” *arXiv:1504.01942 [cs]*, Apr. 2015, arXiv: 1504.01942. [Online]. Available: <http://arxiv.org/abs/1504.01942>
- [15] T. Lin, M. Maire, S. J. Belongie, L. D. Bourdev, R. B. Girshick, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: common objects in context,” *CoRR*, vol. abs/1405.0312, 2014. [Online]. Available: <http://arxiv.org/abs/1405.0312>
- [16] A. Milan, L. Leal-Taixé, I. Reid, S. Roth, and K. Schindler, “MOT16: A benchmark for multi-object tracking,” *arXiv:1603.00831 [cs]*, Mar. 2016, arXiv: 1603.00831. [Online]. Available: <http://arxiv.org/abs/1603.00831>
- [17] MMDetection Contributors, “OpenMMLab Detection Toolbox and Benchmark,” <https://github.com/open-mmlab/mmdetection>, 8 2022, accessed: 2022-12-07.
- [18] MMOCR Contributors, “OpenMMLab Text Detection, Recognition and Understanding Toolbox,” <https://github.com/open-mmlab/mmcocr>, 8 2020.
- [19] OpenMMLab Contributors, “OpenMMLab Github Overview,” <https://github.com/open-mmlab>, 2023, accessed: 2023-02-03.
- [20] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Z. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” *CoRR*, vol. abs/1912.01703, 2019. [Online]. Available: <http://arxiv.org/abs/1912.01703>
- [21] tensorflow.org, “Xception documentation,” https://www.tensorflow.org/api_docs/python/tf/keras/applications/xception/Xception, 2022, accessed: 2022-12-07.
- [22] O. Urbann, O. Bredtmann, M. Otten, J.-P. Richter, T. Bauer, and D. Zibriczy, “Online and real-time tracking in a surveillance scenario,” *ArXiv*, vol. abs/2106.01153, 2021.
- [23] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Online: Association for Computational Linguistics, Oct. 2020, pp. 38–45. [Online]. Available: <https://www.aclweb.org/anthology/2020.emnlp-demos.6>
- [24] Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick, “Detectron2: A pytorch-based modular object detection library,” ai.facebook.com/blog/detectron2-a-pytorch-based-modular-object-detection-library/, 2019, accessed: 2022-10-13.
- [25] —, “Detectron2-repository,” <https://github.com/facebookresearch/detectron2>, 2019.